

Technical Companion

Questions and answers from the project discussions: statistical power, what the saved artifacts contain, how the two-tier system flows, and the geometry underneath it

1. Statistical power: which ratio governs the fit

Q. 10,000 passages give ~10 samples per dimension?

Yes. 10,000 passages become 7,500 training rows after stage A3's 75/25 split, and $7,500 / 768 = 9.8$ **rows per input dimension**. Run 1 was $1,234 / 768 = 1.6$.

The subtlety is which denominator is correct. A 768x768 map holds roughly 590,000 parameters, but that number does not set the requirement. Multi-output ridge has the closed form $W = (X'X + \alpha I)^{-1} X'Y$, in which the inverted matrix depends on the inputs alone -- the outputs never enter it. The problem therefore decomposes into 768 independent regressions, one per output dimension, each estimating 768 coefficients from the same design matrix and the same shared inverse. They neither interact nor compete for data. The statistical question is asked once, per output regression: are there enough rows to estimate 768 coefficients? Against the conventional minimum of 5 to 10 rows per coefficient, run 2 passes comfortably and run 1 fails badly.

Run	Passages	Training rows	Rows / input dim	Rows / 590K params
1	1,646	1,234	1.6 (starved)	0.002 (misleading)
2	10,000	7,500	9.8 (adequate)	0.013 (misleading)

Table 1: The governing ratio is rows per input dimension. Dividing by the total parameter count would suggest the fit is hopeless; it is 768 tractable problems, not one intractable one.

A useful consequence: output width is nearly free. The Experiment B adapter maps 512 dimensions into 768, so its requirement scales with **512**, not 768 -- which is why 3,000 image pairs sufficed there.

2. Reading the A1 artifact

Q. What does `activations.npz` with `A_layers [L, N, d]` actually contain?

Three arrays, one per model that processed the sentences: **A_layers** from GPT-2, **B_layers** from Pythia-160M, **R_layers** from the random-weights control. Each is a three-axis cube: **L** = 13 layers (12 transformer blocks plus the embedding output, which counts as layer 0), **N** = 10,000 sentences, **d** = 768 hidden dimensions. The executed run produced (13, 10000, 768) for all three.

One element, concretely: `A_layers[5, 200]` is a 768-dimensional vector -- GPT-2's layer-5 representation of sentence 200, mean-pooled over that sentence's tokens. `A_layers[5]` alone is a 10,000 x 768 matrix: the whole dataset as seen by layer 5.

The shape exists to serve two downstream operations. A2 takes `A_layers[i]` and `B_layers[j]` -- two 10,000 x 768 matrices -- and compares their internal similarity structures, for all 13x13 pairs. A3 takes matched layers, splits the 10,000 rows 75/25, fits a ridge map on 7,500 and measures R-squared on 2,500. Both need 'same sentences, one vector each, per layer', which is exactly [L, N, d]. Critically, row *i* is the same sentence in all three arrays; that alignment is what the stage asserts before saving.

Size note: three float32 arrays of 13 x 10,000 x 768 come to about 1.2 GB, and `savez_compressed` barely compresses dense floats. Set `DATA_DIR` to a Drive path, or the file dies with the VM.

Q. Why move the tensors from CUDA to numpy on CPU?

Four reasons, all in one line of `extract()`. **GPU memory lifecycle**: pooled results would grow to ~400 MB per model while the model itself is resident, and would still hold VRAM when the next model loads -- the

del/empty_cache pattern between models only works if nothing else holds memory. **The downstream work is not GPU work:** linear CKA reduces to operations on 768x768 matrices and A3's ridge is a single solve of a 768x768 system, milliseconds on CPU; transfers would cost more than they save. **Persistence:** savez takes numpy arrays, and stages run as separate notebooks, possibly separate VMs, so the boundary between stages must be a CPU-side file. **Dtype hygiene:** .float() guarantees float32 storage even if a model loads in fp16, giving the ridge solve stable numerics. The one cost is a CUDA synchronization per batch, which is noise against a transformer forward pass. The opposite choice would be right for an iterative GPU fit -- which is why B2's MLP comparison does keep tensors resident.

3. What the activations are

Q. What do the extracted activations actually represent?

The models' internal states -- what each network computed while reading the sentences. Every transformer block outputs one 768-d vector per token; A1 averages those over the sentence's real tokens into one vector per sentence per layer. Weights are the *how* of computation and are incomparable across models because of permutation symmetry; activations are the *what*, computed on shared inputs, which gives the two models a common reference point. The hypothesis is a claim about activations, so this is the step that makes it measurable.

No individual dimension carries meaning -- dimension 412 is not 'past tense' or 'about animals'. Meaning lives in geometry: which sentences land near which others. That is why every measurement in the project is a relation (cosine, CKA, rank) rather than a coordinate reading.

Depth matters, which is why all 13 layers are saved rather than only the last. Layer 0 is near-lexical, being the embedding table, so sentences sharing words look similar regardless of meaning. Middle layers are where abstraction lives and where similar *situations* cluster even without vocabulary overlap. Late layers re-specialize toward each model's own output head and its own tokenizer's vocabulary -- which is why both endpoints score lower in the results while the middle peaks at layer 11.

The control array is the same procedure through untrained weights. It is not noise: random transformations still propagate input structure, and a random embedding table is still *consistent*, so layer 0 retains real similarity (R-squared 0.375 there). What it lacks is constructed abstraction, which is why its usefulness decays with depth while the trained curve rises.

Q. How are the three models created, and why do the arrays match?

Only two models are loaded; the third is constructed. The distinction is `from_pretrained` versus `from_config`: a config is a blueprint (layer count, hidden width, head count, initializer scale), and `from_pretrained` builds from the blueprint *and then loads the checkpoint*, while `from_config` builds and stops, leaving weights at their random initialization. That is Pythia at second zero of training -- identical architecture, identical input pipeline, no learning. Seeding matters: without it the control changes every run.

Array	Constructed by	Weights
A	<code>AutoModel.from_pretrained('gpt2')</code>	OpenAI's trained weights (WebText, 2019)
B	<code>AutoModel.from_pretrained('EleutherAI/pythia-160m')</code>	EleutherAI's trained weights (The Pile, 2023)
R	<code>AutoConfig.from_pretrained(...) -> AutoModel.from_config(...)</code>	first 13 layers random values, zero training steps

Table 2: The control is matched to B, not A, so that comparing A-to-B against A-to-R isolates exactly one variable: whether B's weights ever learned anything.

The identical shapes have three different causes. **13 layers** and **768 dimensions** are coincidences -- different labs and years both chose conventional values for the ~120-160M parameter class. **10,000 rows** is by construction, since the same sentence list is passed to all three extractions, and that is the one that matters. The coincidences are convenient rather than load-bearing: they let A2's heatmap be square with a meaningful diagonal and A3's matrix be square, but CKA is dimension-agnostic and a ridge map between unequal widths is simply rectangular -- exactly as in the 512-to-768 adapter. Identical shapes emphatically do not mean identical values; the raw cosine between GPT-2's and Pythia's vectors for the same sentence is near zero, because the coordinate systems are unrelated. That a single matrix reconciles them anyway is the finding.

4. Embeddings for retrieval

Q. What is the semantic meaning of a RAG embedding?

A distinction this project makes vivid: the vectors A1 extracted are **not** RAG embeddings, though both are transformer activations. A dedicated embedding model is trained contrastively -- matching query/document pairs pulled together, mismatched pushed apart -- which explicitly optimizes the property retrieval needs: semantic similarity becomes geometric proximity, so cosine distance means 'about the same thing'. A generative model's hidden states are optimized for next-token prediction. They carry rich semantics, as Experiment A proved by stitching them, but their geometry was never shaped so that distance means relatedness.

As with activations, meaning lives in relations rather than coordinates, from which it follows that an embedding is meaningful only relative to other embeddings *from the same model* -- precisely the problem the adapter solves for the phone/server split. Four practical consequences: chunk size is a semantic decision, since one vector must summarize the whole chunk and an over-long chunk averages several topics into a blurry centroid; query and document are asymmetric, which is why models such as E5 use query/passage prefixes; absolute cosine values are calibration-dependent, so a fixed threshold is meaningless across models and gaps or ranks should be read instead; and embeddings cannot represent negation, since 'contract without an indemnity clause' embeds almost identically to its opposite.

The connection back to this work: because embedding spaces converge in the way measured here, RAG embeddings are translatable. A linear adapter can map a cheap local embedder into a premium model's space, allowing local indexing with premium-space querying -- the EmbeddingAdapters and Drift-Adapter pattern, and the reason the convergence result matters for text retrieval and not only for an image index.

5. Data flow of the adapter and the shared index

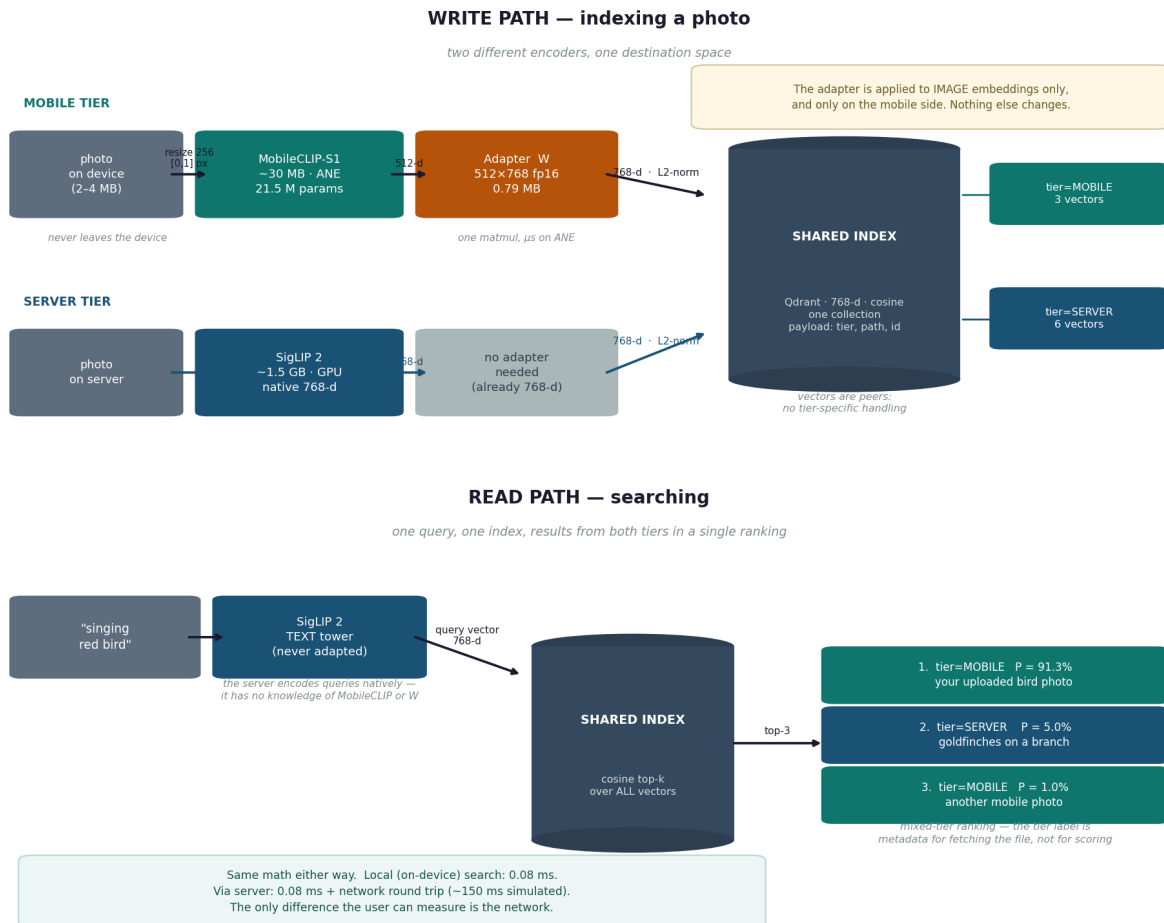


Figure 1: Write path (indexing) and read path (search).

Writing from the phone. Photo -> preprocess (resize 256, center-crop, raw [0,1] pixels, no normalization) -> MobileCLIP-S1 gives 512-d -> multiply by W (fp16, 0.79 MB, one matmul on the Neural Engine) -> 768-d -> L2-normalize -> write to Qdrant with payload tier=MOBILE. The photo never leaves the device; only the 3 KB vector syncs. Measured at 39 ms end to end.

Writing from the server. Photo -> SigLIP 2 -> 768-d natively -> L2-normalize -> write with tier=SERVER. No adapter is involved, because the server's encoder defines the target space. This asymmetry is the design's economy: translate the cheap side into the expensive side's coordinates, not both into a third space.

Searching, either way. Query text -> SigLIP's text tower (never adapted, unaware MobileCLIP exists) -> 768-d query vector -> cosine top-k over the single collection. Nothing branches on tier: the index computes cosines and returns whatever is closest, so a phone-indexed photo and a server-indexed photo compete in one ranking. The tier tag is metadata telling the app where to fetch the file afterwards -- device storage or object store -- and plays no part in scoring. Search latency is identical for both tiers (0.08 ms measured); the 150 ms server figure is an explicitly simulated network round trip.

Q. What is a 'text tower'?

In dual-encoder models one tower is the image network and the other the text network; they are separate models with separate weights and different internals, joined only by terminating in the same output space. Contrastive training is what put them there: matched image-caption pairs pulled together, mismatched pushed apart, so the two towers were co-trained to agree. In this system the text tower needs no adaptation because it already speaks the target space natively -- it *defines* half of it. That obliviousness is the elegance: one side is adapted, the other needs no change, no retraining, no awareness.

6. The geometry underneath

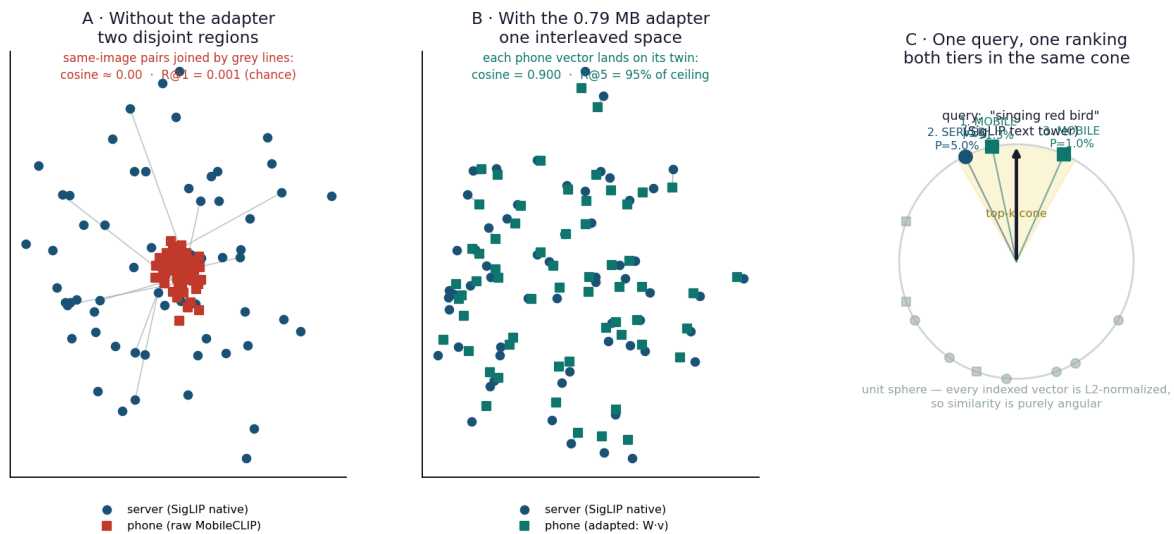


Figure 2: Similarity geometry of the shared index. Points are illustrative but constructed to reproduce the project's measured statistics: adapted-to-target cosine 0.900, raw cross-space cosine ~ 0.00 .

Panel A shows the two encoders' outputs before adaptation, with grey lines joining *same-image* pairs. The lines are long: MobileCLIP's vector for a photo and SigLIP's vector for the same photo are strangers, cosine near zero -- which is why cross-tier search scored exact chance ($R@1 = 0.001$). **Panel B** applies one matrix to the same photos: each phone vector lands essentially on its server twin and the two clouds interleave, which is the geometric meaning of a shared index -- there is no region one could circle and call 'the mobile part'.

Panel C shows retrieval. Because every vector is L2-normalized, all of them live on the unit sphere and similarity is purely angular: a query defines a direction, and the top-k are whatever falls in the narrowest cone containing k vectors. The measured demo ranking is drawn in -- MOBILE at 91.3%, SERVER at 5.0%, MOBILE at 1.0% -- all inside one cone, with nothing in the geometry distinguishing tiers.

This panel also resolves an apparent contradiction from stage B2: the adapter's R-squared of 0.592 and its cosine of 0.900 are not in conflict, because R-squared penalizes magnitude errors that L2-normalization discards, while cosine measures the only thing the index ever asks about -- direction.

7. Layer correspondence, and why the full matrix matters

Q. Why only 12 hidden layers?

Because both models are the entry tier of their families, and depth is the main axis those families scale along: GPT-2 runs 12 / 24 / 36 / 48 blocks at widths 768 / 1024 / 1280 / 1600. The value 12 is the BERT-base convention, inherited by GPT-2 small and ViT-Base, and it falls out of a parameter budget -- a block costs roughly $12d^2$ parameters, so at $d=768$ that is $\sim 7.1M$ per block, $\sim 85M$ for twelve, plus a $\sim 38.6M$ embedding table for GPT-2's vocabulary: the familiar 124M. Choosing 12 layers at width 768 is choosing a $\sim 100M$ -parameter model. Depth is not increased at fixed width because layers are sequential and add latency that cannot be parallelized, while width fills matmuls efficiently; deep-and-narrow also trains less stably. Note the terminology gap behind the 12-versus-13 in the saved arrays: 12 counts transformer blocks, while `output_hidden_states` returns 13 tensors because the embedding output counts as hidden state 0.

Two consequences for this experiment, both favourable. Twelve layers is enough depth for the phenomenon under test to appear -- the stitching curve climbs from 0.59 at layer 0 to 0.80 at layer 11. And because the Platonic Representation Hypothesis predicts alignment rising with capability, models of this size sit at the *low* end of the curve, so the measured 0.74-0.80 was obtained in the least favourable regime.

Q. Why is matching layers by index not a weaker method than matching by relative position?

It is not weaker -- an earlier formulation of this overstated the difference, and the correction is worth recording. When depths are equal, index matching *is* relative matching: layer 5 of 12 and layer 5 of 12 both sit at 42% depth, so the two criteria coincide and no choice has to be defended. They diverge only when depths differ, and then the real issue is not accuracy but a researcher degree of freedom: mapping layer i of 12 to $\text{round}(i \times 23/11)$ of 24 assumes computation distributes proportionally across depth, which is plausible but untested, and rounding forces arbitrary calls. Different defensible mappings produce different-looking diagonals.

There is a third option, better than either, and it is what the CKA literature does: do not pre-commit to a mapping at all. Compute the full rectangular matrix and let each row's argmax reveal which layers correspond. The correspondence becomes a finding rather than an assumption. The accurate ranking is therefore **data-driven (argmax) $\geq$ index (when depths match) $>$ assumed proportional (when they do not)**.

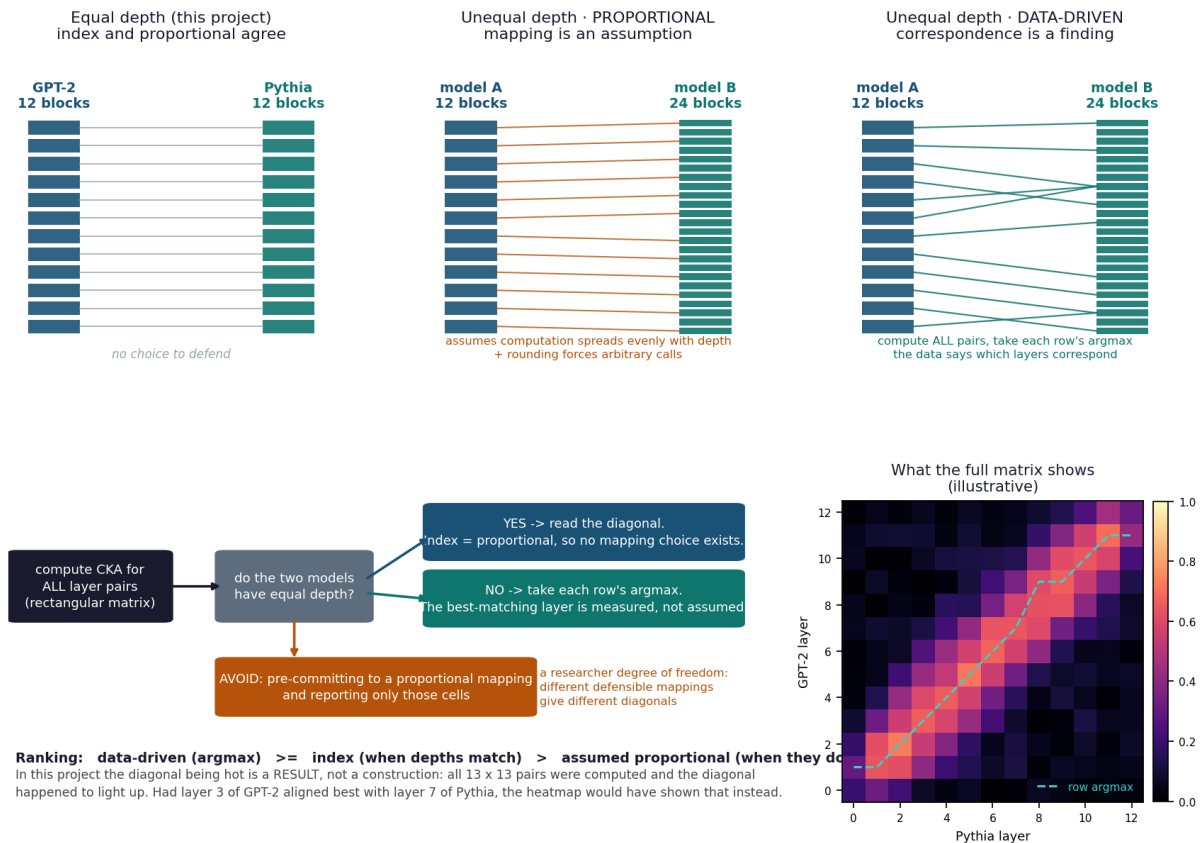


Figure 3: Three correspondence strategies, the decision flow, and what the full matrix reveals that a pre-selected diagonal cannot.

Q. Why is 'we computed all pairs and the diagonal lit up' a stronger claim than 'we matched by index and the numbers were high'?

Because the two sentences differ in what could have refuted them, and a claim's strength is measured by what it forbids. The first formulation describes 13 measurements: pairs chosen in advance, looked at, found large. That is real evidence that corresponding layers are similar, but it is silent about the surrounding 156 cells. Every cell might be high -- meaning the layers are indiscriminately similar, a general 'both encode English' effect with no structure. Or the true best match for GPT-2's layer 3 might be Pythia's layer 8 -- meaning the models process at different rates and the index pairing was systematically off-target while still scoring decently. Neither possibility is excluded by the numbers that formulation would report.

The second describes 169 measurements, of which 13 were predicted high and 156 predicted lower. It could have failed visibly: a uniformly bright square (no structure), a shifted band (offset correspondence), a blocky pattern (interchangeable chunks of layers). It failed in none of them. The information is not in the diagonal's height but in the **contrast** between the diagonal and everything else. High diagonal values say the models are similar; high diagonal values *relative to off-diagonal* say they are similar in a depth-ordered way -- early to early, middle to middle. Only the second supports the interpretation the report actually makes, that both models build abstraction progressively and in the same order. The off-diagonal cells license that reading, and they exist only if computed.

The same structure recurs throughout the project, which is why it is worth naming. R-squared of 0.737 alone is a number; 0.737 against a control at 0.173 is evidence. The MLP's cosine of 0.903 alone is unremarkable; 0.903 against the linear map's 0.900 is a finding about linearity. A rank-1 retrieval in the demo alone is a screenshot; rank-1 alongside a negative control that stays flat is a working system. In each case the second measurement is the one that could have come out otherwise -- which is exactly where the information lives.

8. Dimensionality and the bottleneck

Q. What does 768-d mean, and why can't the phone encoder keep 1024?

768-d means a vector of 768 numbers -- the width of SigLIP 2's embedding space and therefore of the shared index. The value is a convention of the ViT-Base and BERT-base tier (12 heads x 64 dimensions), the same width that appears in both language models of Experiment A. Three widths matter here: MobileCLIP emits **512**, SigLIP emits **768**, and the adapter is therefore **512 x 768** -- literally input width by output width, which is why a linear map can bridge encoders of different dimensionality where a rotation cannot.

On the bottleneck: MobileCLIP-S1 *does* compress 1024 to 512. Its trunk widens to 1024 channels in the final convolution, pools, and then a single Linear(1024 -> 512) emits the embedding. A linear map from 1024 to 512 has a null space of at least 512 dimensions: two images differing only along those directions produce literally identical outputs. No downstream function -- linear, MLP, or enormous transformer -- can separate them afterwards, because the difference is not degraded but absent.

This is not a flaw. The projection is trained, so it preserves the directions that matter for the contrastive objective and discards the residue, and 512 was chosen deliberately for bandwidth, index memory and search speed. But it bounds what the adapter can do: expanding 512 to 768 cannot create information, so when SigLIP distinguishes two images that MobileCLIP does not, no adapter recovers the difference. The MLP experiment measured exactly this, gaining +0.003 for ten times the parameters, which partitions the residual gap cleanly -- it is missing input, not translation error. The prediction that follows, now recorded in the report, is that a wider-output sibling should recover most of the gap at the cost of a refit measured in seconds. One honest caveat: the bottleneck is the most localizable explanation, not provably the only one, since MobileCLIP is also a weaker encoder throughout -- 21.5M parameters against SigLIP's several hundred million. The 512-d projection is where the loss is visible and measurable; upstream capacity limits are real but diffuse.